

How the app turns satellite noise into a pace you can trust

A plain-language tour of the app's construction and the arithmetic behind every number on screen — written so the "why" behind a jumpy split or a slow-to-appear speed is never a mystery.

Platform Android & iOS, one codebase **Phase** 1 of 3 — live tracking & GPX logging

Reading time ~10 min

01 Executive summary

Simple Activity Tracker is a **Flutter** app — one Dart codebase that compiles to a native Android and a native iOS app, rather than two separate projects kept in sync by hand. It reads your phone's GPS roughly once a second, turns that stream of coordinates into speed, distance, pace and kilometre splits, and writes the raw route to a GPX file you could open in any mapping tool.

The distinctive engineering problem here is not the arithmetic — distance and speed are two lines of trigonometry — it's that **consumer GPS occasionally lies**. A phone can report a location half a kilometre from where you actually are, for one sample, before correcting itself. Unfiltered, that single bad sample would spike your speed to over 1,000 km/h and permanently inflate your total distance. Most of what follows is the app deciding, sample by sample, whether to believe what the GPS just told it.

02 Technology & construction

Four building blocks, each doing one job. None of them talk to the phone's GPS hardware directly except one — which is deliberate.

FRAMEWORK

Flutter / Dart

Google's UI toolkit. One source tree compiles to native ARM binaries for both Android and iOS — no shared web view, no separate Swift/Kotlin apps to maintain in parallel.

GPS ACCESS

geolocator

The one package allowed to talk to the OS location APIs directly. Tuned per-platform: `bestForNavigation` accuracy, background updates enabled on iOS via an `Info.plist` entry. On Android it also asks for the phone's location hardware directly rather than the network-assisted provider Android normally prefers — that provider can stall waiting on Wi-Fi even when GPS itself is on and working, which otherwise left a run stuck on "Acquiring GPS..." with Wi-Fi turned off.

STATE MANAGEMENT

Riverpod

A one-way data pipeline: the GPS stream feeds a controller, the controller emits an immutable state object, and every widget on screen just redraws from that. No component reaches backward to mutate another's data.

TRACK STORAGE

GPX 1.1 files

The open standard used by Garmin, Strava, and most mapping software. Every run is written to disk as a standard `.gpx` track — portable, and readable outside this app.

The data path, GPS fix to screen

HARDWARE

Phone GPS radio

Combines satellite, Wi-Fi and cell-

CORE/LOCATION

GeolocatorLocationService

Wraps the OS location

DOMAIN/TRACKING

MetricsEngine

Pure calculation, no UI

FEATURES/LIVE_RUN

LiveRunController

Riverpod controller. Also drives

UI

Metric tiles

Re-render automatically whenever

tower signals into one position estimate, roughly once per second.

→

API. Nothing outside this file ever sees a raw platform type — the rest of the app only knows a simple lat/lon/speed/accuracy shape.

→

awareness. Accepts or rejects each fix, accumulates distance and time, detects kilometre crossings.

→

the GPX writer and the screen-stays-on lock in parallel with the metrics engine.

→

the controller's state changes. No polling, no manual refresh calls.

Why the layers are separated

core/location

The *only* place that imports the GPS plugin. Swapping GPS providers, or feeding it recorded test routes, touches one file.

domain/*

Plain Dart — no Flutter, no UI import allowed. This is what makes the maths in §3 unit-testable in isolation, run in milliseconds, with no phone or simulator needed.

features/live_run

The screen and its controller. Reads from `domain`, never the reverse — the calculation layer has no idea a screen exists.

This isn't architectural tidiness for its own sake — it's what let the outlier-rejection logic in §3 be tested against thousands of synthetic GPS traces with zero live GPS involved, and what let this session catch and fix three separate bugs before they were ever tested against your own runs.

03 How each metric is actually calculated

The phone hands the app a raw stream of (latitude, longitude, timestamp, accuracy) readings. Everything you see on the tracking screen is derived from that stream — nothing is estimated from your step count or phone motion sensors.

How often a fix arrives

REQUESTED INTERVAL	REQUESTED PRECISION	DISTANCE FILTER	TYPICAL REAL INTERVAL
1 s (Android)	best -for-navigation	0 m — no fix skipped	~1–3 s, terrain-dependent

One second is a *request*, not a guarantee — the OS and the GPS chipset decide the real cadence, and it varies with sky visibility. Open sky and a good lock: close to 1 fix/second. Under tree cover, between tall buildings, or on a cold start: gaps of several seconds are normal, and the app is built to tolerate them without the display freezing or jumping.

The core geometry

Every distance calculation starts with the **haversine formula** — the standard way to find the straight-line distance between two latitude/longitude points on a sphere. Given any two consecutive fixes, it returns the distance between them in metres; dividing that by the time between their timestamps gives the implied speed for that segment.

Current speed

km/h · min/km

the GPS chip's own reported speed for the latest fix

The big number on the tracking screen reads the phone's GPS chipset's own speed measurement directly, not something the app derives from position — it isn't smoothed and isn't passed through any of the §4 filters, so it's the reading most exposed to a single noisy fix. If the chip doesn't report a speed for a given fix, it falls back to distance ÷ time between that fix and the previous one.

Average speed

km/h · min/km

total distance ÷ total moving time
– since Start, excluding pauses

Cumulative across the whole run. A discarded bad fix (see §4) contributes nothing here — the average only reflects distance the engine actually accepted.

Distance

km, 2 d.p.

$\sum \text{haversine}(\text{fix}_n, \text{fix}_{n+1})$
– summed segment by segment

Split pace

current km in progress

distance into this km ÷ time
into this km, live

Built incrementally, one fix at a time — the app never needs to replay the whole run's history to update this number.

Resets each time you cross a whole kilometre. The crossing moment itself is *interpolated* — see below — so it doesn't wait for the next GPS fix to land exactly on the line. If this split has a target, the tile turns green or red — see "Split targets" below.

Last split

m:ss for that km

time at which the previous
km boundary was crossed

Because the crossing time is interpolated between two real fixes, this is accurate to a fraction of a second — not rounded to whichever fix happened to land nearest the line. If that split had a target, this tile is also tinted by whether it was met.

Time

h:mm:ss

wall clock since Start
minus any time spent paused

This **is** a wall clock — it counts up whether or not GPS is producing usable fixes, so it never freezes if you're stood still or signal briefly drops out. It only stops advancing while genuinely paused; Average speed and the splits below use a separate *moving* time internally, which does exclude time spent stationary or filtered out by §4 — that's why Average speed can look faster than distance ÷ this Time tile would suggest.

On that interpolation: say you're at 970 m at one fix and 1,050 m at the next, one second later. The 1 km mark falls 75% of the way through that segment, so the app estimates you crossed it 750 ms into that one-second gap — rather than crediting the whole split to whichever fix happened to land on either side of the line. This is also why a split time can differ slightly from what a simpler tracker would show for the identical GPS data.

Split targets

Optionally, each split can carry a **target pace or speed**, set on the Splits screen before a run. Splits can either repeat at one fixed size ("every 1 km", optionally all sharing one target) or follow a custom plan of individually sized splits with their own targets — an interval session of, say, 90 seconds fast then 60 seconds easy.

The Split pace tile is judged on the **current split's average pace so far** by moving time — the same number the tile already shows — never on the instantaneous speed reading above it, which is too noisy fix-to-fix to colour a tile with. It turns green when that average is within 5% of the target either way, and red outside that band, with an arrow showing which way to adjust: pointing down means slow down, pointing up means speed up. To

stop every split opening red for its first few seconds before pace has settled, there's a brief grace period — about ten seconds of moving time — before any colour shows at all. A completed split's own tile and its row in the splits list are tinted the same way, judged on that split's final average with no grace period, since the number is already settled.

If a custom plan's own splits run out before the activity ends, tracking simply continues at the plan's base split size with no target — nothing stops working, the colours just switch off.



04 Precautions against bad GPS data

GPS receivers occasionally hand back a wrong answer — most often right at the start of a run, before the chipset has a confident lock, or indoors, where reflected signal ("multipath") can make a stationary phone look like it's slowly moving. Every fix passes through several checks before it's allowed to count, and the checks differ slightly for running versus cycling.

GUARD	WHAT IT CATCHES	THRESHOLD
Measured-accuracy filter	Some fixes carry an accuracy figure the OS never actually measured — just a placeholder. Trusting that value as if it were a confident reading would let an arbitrarily bad fix straight through the next filter, so it's rejected outright, regardless of what the number says.	reject if <code>accuracy unmeasured</code>
Accuracy filter	The OS reports a confidence radius with every fix. A fix the phone itself isn't confident about is dropped before it reaches any calculation.	reject if <code>accuracy > 25 m</code>

GUARD	WHAT IT CATCHES	THRESHOLD
Plausible-speed filter	No one sustains this pace for the chosen activity. A fix implying it is treated as a glitch — a stale or default location before the GPS gets a real lock — not real motion. Cycling's cap is higher, since a fast descent is routine on a bike but not on foot.	reject if implied speed > 12 m/s (43 km/h) running 25 m/s (90 km/h) cycling
Absolute-jump cap	Closes a loophole in the speed filter alone: given a long enough gap since the last fix, <i>any</i> distance starts to look like a slow, plausible pace. This catches a large teleport regardless of how much time has passed. Cycling's cap is larger, since a bike can cover far more ground in a genuinely sparse-fix gap (a tunnel, a backgrounded phone) without it being a bad fix.	reject if jump > 20 km running · 40 km cycling
Stationary gate	Indoors, reflected/multipath fixes routinely wander a few metres between updates while the phone doesn't move at all — implying a low but non-zero speed, well within even the plausible-speed filter's limits, so position data alone can't reliably tell that apart from a real slow walk. Instead the app reads the GPS chip's own speed measurement for each fix — which comes from the radio signal itself, not from comparing positions, so multipath that shifts the reported <i>position</i> doesn't produce a matching false <i>speed</i>. Three consecutive fixes reporting real motion are required before a segment starts counting; a single fix already moving is enough to stop counting again, so a genuine stop is recognised immediately. If the chip doesn't report a speed for a fix, the app falls back to a simple	need 3 fixes ≥ 0.6 m/s to start; < 0.4 m/s to stop (fallback: ignore if segment < 1.2 m)

minimum-segment-size check
instead.

What happens to a rejected fix. It's discarded from the live metrics — distance and average speed act as though it never arrived. But an implausible-speed or absolute-jump rejection isn't simply forgotten: if the *next* fix agrees with the rejected one (i.e. two consecutive readings that are mutually consistent, showing at least a couple of metres of real movement between them), the app concludes the earlier "good" fix was actually the mistake, and quietly re-anchors its tracking onto the new, more plausible position — crediting no extra distance or time for the jump itself, the same way a pause and resume works. That candidate has to be confirmed within 30 seconds, or it's treated as a genuine gap rather than a recoverable glitch.

That recovery step exists because the obvious alternative — always trusting the last accepted fix — has a sharp failure mode: if that fix turns out to have been the bad one, *every subsequent real fix* near your actual position also looks like an impossible jump and gets rejected too, until enough time passes for the maths to catch up. An early version of this logic quarantined 77 seconds of perfectly good running data this way before the fix below it.

On the stationary gate specifically: requiring three consecutive fixes to agree, rather than trusting a single reading, exists because a real capture showed GPS chip noise doesn't always arrive as one clean spike — occasionally it rings for two consecutive fixes before settling back down. A brief noise event can still credit a small amount of phantom distance (typically under a couple of metres, from real indoor test captures) before the gate catches up, but nothing beyond that: once a reading genuinely drops below the stop threshold, crediting halts immediately, with no 3-fix delay in that direction.

The GPX file written to disk, by contrast, keeps **every** fix, filtered or not — filtering is a display decision the app makes for you live; the underlying recording stays complete, in case you ever want to re-process it differently.



What to expect in the real world

GPS accuracy on a phone is a function of sky visibility, not the app. These are realistic bands for a modern smartphone, not lab conditions.

OPEN SKY · CLEAR CONDITIONS

Accuracy typically **$\pm 3\text{--}8\text{ m}$** . Speed and pace settle within a few seconds of starting and track smoothly. This is the everyday case for most road and park running.

URBAN CANYON · TALL BUILDINGS

Signal reflects off buildings before reaching the phone ("multipath"), inflating accuracy to **$\pm 15\text{--}30\text{ m}$** . Pace readings get noisier; more fixes sit near the 25 m accuracy cutoff and get dropped.

TREE COVER · NARROW TRAILS

Fix interval stretches — expect gaps of **$3\text{--}8\text{ s}$** between updates rather than a steady 1 s. Distance still accumulates correctly; the current-speed reading simply updates less often.

FIRST 10–20 SECONDS OF ANY RUN

The GPS chipset is still narrowing its lock. This is precisely the window the outlier filters in §4 exist for — an early bad fix here is the most common real-world trigger.

A rule of thumb: if a single split or momentary speed reading looks implausible, distance and average speed are almost always still correct — the filters described in §4 exist specifically to protect those cumulative numbers, even when one input sample was briefly wrong. It's the instantaneous readings — current speed, in-progress split pace — that are most exposed to real-time GPS noise, by nature of updating every second rather than averaging over the whole run.